Software Engineering — Digital Assignment 3 / Review 3

PreMerge

Project Report

A Git History-Based Predictive System for Early Merge Conflict Detection in Collaborative Software Development.

Project team	
Vikram (`@Waterbottles792`)	Core engine, CLI, GitHub Action
Kanika Rathore (`@kanika10-hub`)	Documentation & planning
Repository: `github.com/Waterbottles792/PreMerge`	
Register No. / Name / Slot	[to be filled by student]

Table of Contents

Section	Content
1	Problem Statement
2	User Stories
3	System Architecture and System Design
4	Design of Tests
5	Appendix — (a) SRS · (b) DFD · (c) ERD · (d) UML diagrams · (e) Code listing / GitHub link

Abstract

PreMerge predicts merge conflicts *before* they happen. It reads a repository's commit history and live branch diffs, scores every pair of active branches for conflict risk, and explains each score in plain English. Two independent signals are combined: **direct overlap** (do the branches edit the same files and the same line ranges?) and **historical coupling** (do they edit files that have always changed together in the past?). The second signal reveals *hidden logical conflicts* that `git merge` cannot. PreMerge ships as a CLI (`premerge analyze`), a Docker image, and a GitHub Action that posts and updates one risk-report comment on each pull request.

1. Problem Statement

In collaborative development, merge conflicts are discovered at the **worst possible moment** — at merge time, after both sides have finished their work and built on incompatible assumptions. The cost of resolving a conflict grows with the time two branches have diverged, yet developers receive no signal until the divergence is complete.

Limitations of existing tooling

- `git merge` / `git rebase` report conflicts only when the merge is attempted.
- CI merge checks (e.g. "this branch has conflicts") also fire only at merge time and only against the base branch — never branch-to-branch.
- Nothing in the standard toolchain detects **logical coupling**: two branches can merge cleanly and still break because the files they touched are historically interdependent (e.g. `auth.py` and `session.py` co-changed in 9 of the last 10 commits).

Objective

Provide a continuously available, ranked view of which in-flight branches are on a collision course, early enough that the cheap fix — a conversation, a rebase, a re-sequencing of work — is still available.

Vision statement

For software teams working in parallel on a shared codebase, *who* discover merge conflicts only when they are already expensive, *PreMerge is* a git-history analysis tool *that* predicts

and ranks conflict risk between branches days before a merge is attempted. *Unlike* merge dry-runs and CI merge checks, which are reactive and text-level, PreMerge combines live divergence with learned historical coupling and explains its reasoning.

Scope, assumptions and constraints

In scope (v1)	Assumptions / constraints
Pairwise branch risk ranking; file- and line-level overlap; Jaccard historical coupling; weighted score; CLI, JSON, SQLite cache; GitHub Action PR comment.	Standard git repo with full history (no shallow clone); append-only history; reasonably scoped commits; merge commits excluded; file/line-level analysis (no AST); scores are comparative rankings, not calibrated probabilities; Python 3.11+ and `git` on PATH; GitHub Actions only.

Success metrics

Metric	Target	Status
Unit-test coverage of core logic	> 80 %	**96 % achieved**
Runtime dependencies	≤ 3	**2 (click, rich)**
Warm (cached) re-run	< 1 s	Sub-second on the small demo repo (≈0.3 s); not yet benchmarked on large repos
Precision / Recall / F1	≥ 0.70 / 0.75 / 0.72	Not yet measured — requires validation study (future work)

2. User Stories

Personas: **Priya** (developer), **Arjun** (tech lead / reviewer), **Meera** (DevOps), **Dr. Rao** (engineering manager / researcher). The 25 stories are tracked as GitHub Issues #3–#27 with one MoSCoW label each (16 must, 8 should, 1 could).

ID	As a…	I want…	So that…	Pri.
US-3	Priya	compare two branches for merge-conflict risk	I know before merging whether my work overlaps with someone else's	M
US-4	Arjun	every branch pair analysed automatically	I get a full risk picture without specifying pairs	M
US-5	Priya	file-level overlap detection	I can see which files both branches touched	M
US-6	Arjun	line-level overlap detection	I can tell real conflicts from files merely both edited	M
US-7	Arjun	one combined risk score per branch pair	I can rank and sequence merges	M
US-8	Dr. Rao	historical file coupling calculated	hidden conflicts across related files are caught	M
US-9	Dr. Rao	coupling derived from real git history	the signal reflects how the team actually works	M
US-10	Dr. Rao	logically dependent files identified	conflicts hidden behind indirection still surface	M
US-11	Dr. Rao	coupling expressed as Jaccard similarity	relationship strength is quantified	M
US-12	Meera	coupling results cached	repeat CI runs are fast	S
US-13	Priya	the report displayed in the terminal	I can read results without leaving my workflow	M
US-14	Arjun	plain-English explanations per pair	I can quote them in review	S
US-15	Meera	results exported as JSON	other tools and the GitHub Action can consume them	S
US-16	Arjun	branch pairs ranked by risk	I can sequence merges to minimise pain	M
US-17	Arjun	high-risk conflicts visually highlighted	I don't hunt for the ones that matter	S
US-18	Dr. Rao	to configure the scoring weights	I can tune the model to our data	S
US-19	Dr. Rao	configuration loaded from a JSON file	weight changes need no source edits	S
US-20	Meera	to point PreMerge at a repository path	I can analyse repos other than the current directory	S
US-21	Meera	PreMerge to run as a GitHub Action	risk is checked automatically on every PR	M
US-22	Arjun	the report posted as a PR comment	I see it where I make merge decisions	M

US-23	Arjun	the PR comment updated in place	I am not spammed with duplicates	M
US-24	Meera	improved performance on large repos	PreMerge stays usable as history grows	C
US-25	Arjun	every branch compared against main in one command	I get the merge-queue view	S
US-26	Priya	built-in CLI help	I can discover options without external docs	M
US-27	Arjun	a comprehensive combined conflict report	I have one place to review risk	M

Acceptance criteria for key stories (Given / When / Then)

Story	Acceptance criteria
US-6 Line-level overlap	**Given** two branches editing the same lines of `auth.py`, **when** I run `premerge analyze A B`, **then** `auth.py` is reported as "both branches edit overlapping lines" and risk ≥ 0.5. *Verified by* test_cli_pair_json_reports_line_overlap.
US-8/11 Coupling	**Given** history where two files always change together and branches that each touch only one of them, **then** no file overlap is reported, coupling = 1.00 and risk > 0. *Verified by* test_cli_detects_hidden_coupling_without_file_overlap.
US-16 Ranking	**Given** several branches, **when** I run `premerge analyze`, **then** C(n,2) pairs are listed in descending score order. *Verified by* test_cli_all_pairs_ranked_highest_risk_first.
US-12 Cache	**Given** an unchanged HEAD, **then** a second run reads the SQLite cache and returns byte-identical output. *Verified by* test_cache_roundtrip and test_cli_warm_cache_gives_same_result_as_cold.
US-23 In-place comment	**Given** a PR already carrying a comment with the marker `<!-- premerge-report -->`, **when** new commits are pushed, **then** that comment is edited instead of a new one being created. *Verified by* test_main_updates_existing_comment_in_place.

MoSCoW summary

Category	Items
Must (M1–M8)	Branch comparison, file overlap, line overlap, historical coupling, risk scoring, CLI, GitHub Action, PR risk comment — **all implemented**
Should (S1–S10)	JSON output, SQLite cache, configurable weights, explanations, base mode, binary handling, cache bypass implemented; validation study, rename tracking, full SE docs outstanding
Could	Web dashboard (Figma-designed), performance tuning
Won't (v1)	ML-based scoring, AST/semantic analysis, IDE extension

3. System Architecture and System Design

3.1 Architectural style

PreMerge is a **layered pipeline**. A thin interface layer (CLI / GitHub Action) calls a single scoring engine, which orchestrates two independent signal detectors (overlap and coupling). All access to the repository goes through one adapter module (`git_utils`) that shells out to the `git` binary. Modules exchange typed dataclasses only; the module graph is acyclic, so each layer can be tested and replaced independently, and the same pipeline runs unchanged locally, in Docker or in CI.

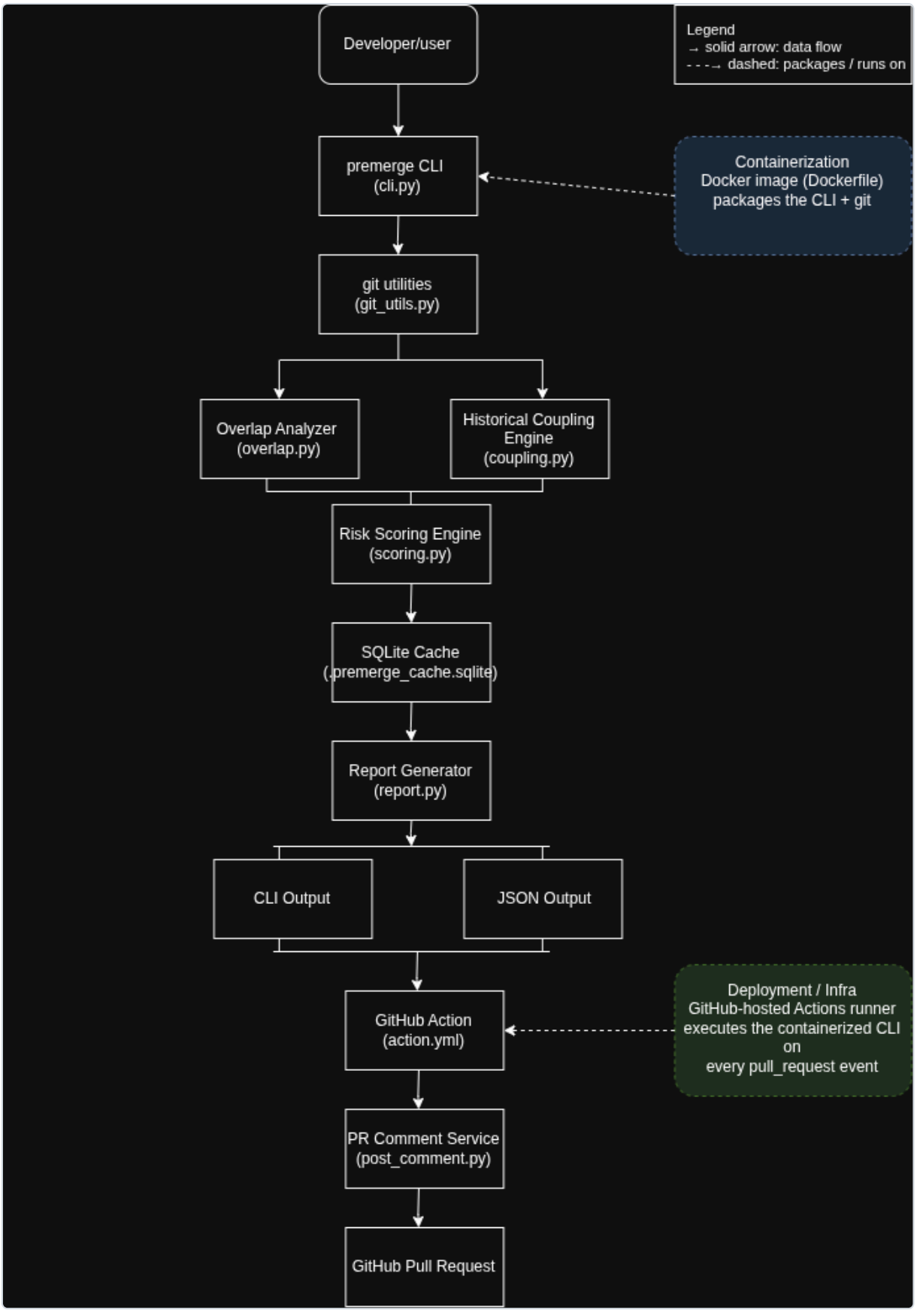


Figure 3.1 — System architecture (with containerisation and deployment), draw.io

Layer	Module	Responsibility
Interface	`cli.py`, `github_action/`	Argument parsing, pair selection (pair / base / all-pairs); PR comment create-or-update

Domain	`scoring.py`	Weighted combination of the signals; plain-English explanations
Signals	`overlap.py`, `coupling.py`	File/line overlap vs merge-base; Jaccard co-change coupling with SQLite cache
Infrastructure	`git_utils.py`, `config.py`, `report.py`	git subprocess wrappers and hunk parsing; weight loading; table/JSON output

3.2 Scoring model

```
risk(A,B) = w1 · line_overlap_score + w2 · file_overlap_score + w3 · historical_coupling

line_overlap_score   = |files with intersecting hunks| / |files touched by both|
file_overlap_score   = |files touched by both|        / |files touched by either|
historical_coupling  = mean Jaccard of (fileA, fileB) pairs where fileA ∈ only-A, fileB ∈
only-B
coupling(f,g)        = commits touching both f and g / commits touching f or g

Default weights: w1 = 0.5, w2 = 0.3, w3 = 0.2  (premerge.config.json)
```

Worked examples (from the demo repository)

Pair	Computation	Risk
feature-a vs feature-b	both edit the same lines of auth.py → line = 1.0, file = 1.0, coupling = 0 → 0.5·1 + 0.3·1 + 0.2·0	0.80 🔴
feature-c vs feature-d	no shared file; auth.py ↔ session.py coupling = 4/4 = 1.0 → 0.2·1.0	0.20 🟡
feature-a vs feature-e	disjoint files, no coupling	0.00 🟢

3.3 Dynamic behaviour

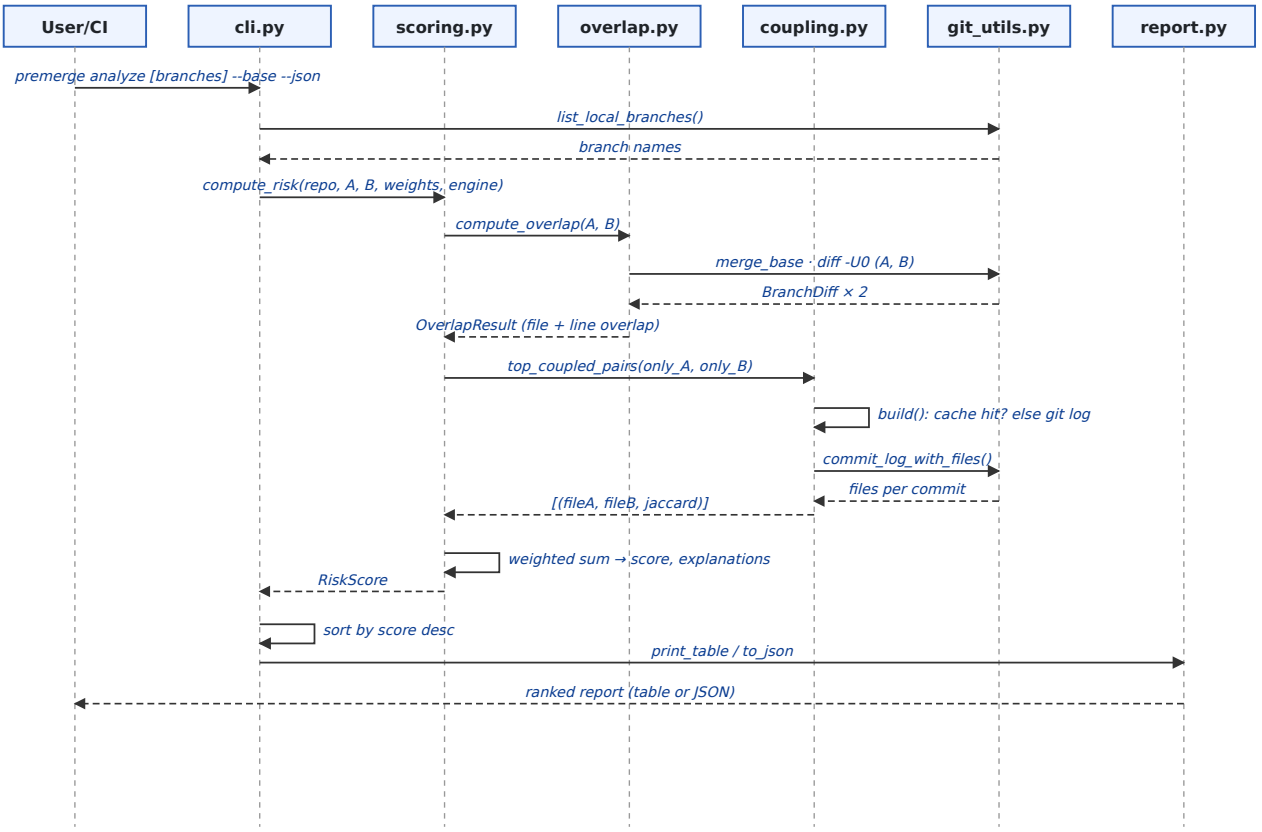


Figure 3.2 — Sequence diagram of one `premerge analyze` run

3.4 Detailed design of key components

- **git_utils.diff_hunks** parses `git diff -U0`: `diff --git` lines give the file, `@@ -a,b +c,d @@` headers give `(start, length)` ranges; a pure deletion counts as length 1; binary files are recorded separately and compared at file level only.
- **overlap._ranges_intersect** — half-open interval test `a.start < b.start+b.len and b.start < a.start+a.len`.
- **coupling.CouplingEngine** — one pass over `git log --no-merges --name-only` builds per-file and per-pair counters; results are serialised to SQLite keyed by `(HEAD hash, rev)`. `top_coupled_pairs` scores only cross-pairs of files each branch touched *exclusively* — the hidden-risk signal.
- **report** — Rich table (red ≥ 0.50, yellow ≥ 0.20, green below) or stable JSON schema.
- **post_comment.py** — stdlib `urllib`; finds the existing comment by the HTML marker and PATCHes it, else POSTs a new one.

3.5 Key design decisions

Decision	Rationale / trade-off
Shell out to `git`, no GitPython	Exact control of diff format; one fewer dependency. Cost: requires git on PATH.
Heuristic scoring instead of ML	No labelled dataset exists yet; a transparent formula is tunable and explainable. ML is future work.
SQLite cache keyed by HEAD	Near-zero repeat cost in CI. Cost: assumes append-only history (`--no-cache` escape hatch).

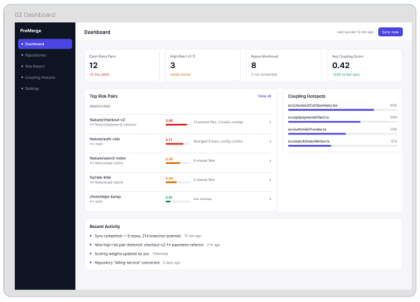
Exclude merge commits from coupling	Their diffs are the union of both parents and would inflate co-change counts.
Layered, acyclic modules + dataclasses	Each module single-responsibility and independently testable.

3.6 Deployment and build

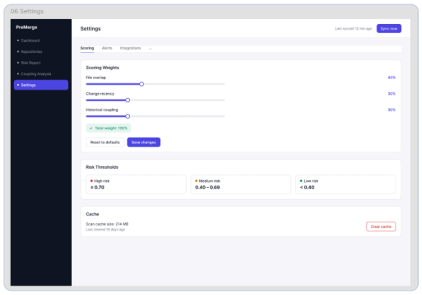
Install via `pip install -e .` (console script `premerge`), run with Docker (`docker run --rm -v "$(pwd)":/repo premerge analyze --repo /repo`), or use the composite GitHub Action. Branching follows GitHub Flow; `tests.yml` runs pytest with coverage on Python 3.11/3.12 and `premerge.yml` dogfoods the tool on its own PRs.

3.7 User-interface design

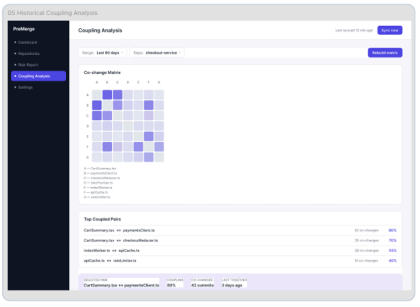
The v1 interface is the terminal table and PR comment. A web dashboard is designed in Figma (6 screens: Login, Dashboard, Repository Selection, Risk Report, Coupling Analysis, Settings) as a proposed extension.



Dashboard



Risk report



Coupling analysis

All-pairs analysis (ranked)

```
$ premerge analyze
```

PreMerge Conflict Risk Report

Branch A	Branch B	Risk	Why
feature-a	feature-b	0.80	auth.py: both branches edit overlapping lines
feature-a	feature-c	0.80	auth.py: both branches edit overlapping lines
feature-b	feature-c	0.80	auth.py: both branches edit overlapping lines
feature-a	feature-d	0.20	auth.py <-> session.py: historically change together (coupling=1.00) — one branch touched auth.py, the other touched session.py
feature-b	feature-d	0.20	auth.py <-> session.py: historically change together (coupling=1.00) — one branch touched auth.py, the other touched session.py
feature-c	feature-d	0.20	auth.py <-> session.py: historically change together (coupling=1.00) — one branch touched auth.py, the other touched session.py
feature-a	feature-e	0.00	no overlap or coupling detected
feature-a	main	0.00	no overlap or coupling detected
feature-b	feature-e	0.00	no overlap or coupling detected
feature-b	main	0.00	no overlap or coupling detected
feature-c	feature-e	0.00	no overlap or coupling detected
feature-c	main	0.00	no overlap or coupling detected
feature-d	feature-e	0.00	no overlap or coupling detected
feature-d	main	0.00	no overlap or coupling detected
feature-e	main	0.00	no overlap or coupling detected

Figure 3.3 — Implemented terminal interface

4. Design of Tests

4.1 Test strategy

Level	Scope	Technique / tool
Unit	coupling, overlap, scoring, git_utils, config, report	pytest on synthetic git repositories created in `tmp_path` with plain `git` calls — hermetic, no network, no checked-in fixtures
Integration	CLI → all modules → git	click `CliRunner` invoking `premerge analyze` against synthetic repos
Regression	Whole suite on every push/PR	GitHub Actions matrix (Python 3.11, 3.12) with coverage
Mutation	All of `premerge/`	mutmut 3.8 — measures fault-detection power of the suite
System / validation	Real conflicting PR	Opened a genuinely conflicting PR: PreMerge scored 0.80 before merge; `git merge --no-commit` confirmed a real content conflict (CHANGELOG, 2026-07-27)

4.2 Test fixture design

The `sample_repo` fixture builds a repo where `auth.py` and `session.py` change together in 4 commits and `unrelated.py` changes alone — giving known ground truth (coupling 1.0 and 0.0). Tests then create branches that overlap, are disjoint, or are only hidden-coupled, using equivalence classes: *same lines / same file different lines / different files coupled / different files uncoupled / binary / deletions / merge commits*, with boundary-value checks on range intersection (overlap by one line, adjacent, containment, disjoint).

4.3 Test case catalogue (30 automated cases)

ID	Test	Level	Verifies	Story
T1	`test_strong_coupling_between_cochanged_files`	Unit	Jaccard = 1.0 for files always co-changed	US-11
T2	`test_no_coupling_for_unrelated_file`	Unit	Jaccard = 0.0 for never co-changed files	US-11
T3	`test_coupling_is_symmetric`	Unit	coupling(a,b) = coupling(b,a)	US-8
T4	`test_cache_roundtrip`	Unit	SQLite file created and re-read correctly	US-12
T5	`test_line_level_overlap_detected`	Unit	Same-line edits flagged at line level	US-6
T6	`test_no_overlap_for_disjoint_files`	Unit	Disjoint branches	US-5

			report no overlap	
T7	`test_overlapping_branches_score_higher_than_disjoint`	Unit	Risky pair outscores safe pair	US-7
T8	`test_ranges_intersect_boundaries`	Regression	Boundary values of range intersection	US-6
T9	`test_pure_deletion_hunk_counts_as_one_line`	Regression	Deletion hunk length floors at 1	US-6
T10	`test_binary_file_is_file_level_only`	Regression	Binary files: file-level only	S6
T11	`test_commit_log_skips_merge_commits`	Regression	Merge commits excluded from coupling	US-9
T12	`test_config_defaults_partial_and_missing`	Regression	Missing / partial config falls back to defaults	US-19
T13	`test_json_report_schema`	Regression	JSON keys and structure stable	US-15
T14	`test_file_overlap_without_line_overlap_scores_lower`	Regression	Same file, distant lines = 0.3 only	US-6
T15	`test_cli_pair_json_reports_line_overlap`	Integration	CLI pair mode end-to-end	US-3
T16	`test_cli_detects_hidden_coupling_without_file_overlap`	Integration	Hidden coupling surfaced with zero file overlap	US-8
T17	`test_cli_all_pairs_ranked_highest_risk_first`	Integration	All-pairs count and descending order	US-4/16
T18	`test_cli_base_mode_compares_every_branch_to_base`	Integration	--base mode pair set	US-25
T19	`test_cli_table_output_renders_title_and_branches`	Integration	Rich table rendering	US-13
T20	`test_cli_no_other_branches_message`	Integration	Graceful message when nothing to compare	US-4
T21	`test_cli_custom_config_changes_score`	Integration	--config weights change the score	US-18

T22	`test_cli_warm_cache_gives_same_result_as_cold`	Integration	Cold, cache-write and cache-hit outputs identical	US-12
T23	`test_table_headers_and_rows`	Unit	Table title, headers and row content	US-13
T24	`test_table_empty_and_no_explanation_messages`	Unit	Empty / no-explanation messages	US-4
T25	`test_table_summarises_extra_explanations_and_lists_them`	Unit	“+N more” summary and detail list	US-14
T26	`test_json_values_roundtrip`	Unit	JSON values and sorted file lists	US-15
T27	`test_comment_body_has_marker_table_and_risk_emoji`	Unit	PR comment marker, table, 🔴🟡🟢 thresholds	US-22
T28	`test_comment_body_empty_report`	Unit	PR comment when nothing to compare	US-22
T29	`test_main_creates_comment_when_none_exists`	Integration	Action POSTs a new comment (mocked API)	US-21
T30	`test_main_updates_existing_comment_in_place`	Integration	Action PATCHes the marker comment, never POSTs	US-23

4.4 Results

Measure	Result
Tests executed / passed	**30 / 30**
Statement coverage	**96 % (config, overlap, report, scoring 100 %; coupling 96 %; cli 93 %; git_utils 92 %)**
Mutation score	76 % overall (447 killed / 587): config 90 %, overlap 87 %, scoring 86 %, git_utils 73 %, coupling 71 %, report 70 %

Full suite + coverage

```
$ python -m pytest -q --no-header -p no:cacheprovider --cov=premerge --cov-report=term-missing 2>&1 | tail -22
..............................                                                                                [100%]
================================================== tests coverage ==================================================
_____________________________________ coverage: platform linux, python 3.13.16-final-0 _____________________________________

Name                     Stmts   Miss  Cover   Missing
--------------------------------------------------------
premerge/__init__.py         1      0   100%
premerge/cli.py             41      3    93%   54-55, 79
premerge/config.py          18      0   100%
premerge/coupling.py        79      3    96%   46, 106, 124
premerge/git_utils.py       64      5    92%   25, 35, 85-86, 105
premerge/overlap.py         29      0   100%
premerge/report.py          30      0   100%
premerge/scoring.py         34      0   100%
--------------------------------------------------------
TOTAL                      296     11    96%
30 passed in 15.26s
```

Figure 4.1 — Test and coverage results

Mutation testing (mutmut 3.8) - per-module score

```
$ python docs/review3/mutation_summary.py; echo; echo 'Sample of results:'; grep premerge docs/review3/logs/mutmut_results.txt | head -6
module        killed  survived  no tests   score
config            37         4         0     90%
coupling          98        40         0     71%
git_utils        128        19        29     73%
overlap           53         8         0     87%
report            71        30         0     70%
scoring           60        10         0     86%
TOTAL            447       111        29     76%

Sample of results:
    premerge.report.x_print_table__mutmut_1: killed
    premerge.report.x_print_table__mutmut_2: killed
    premerge.report.x_print_table__mutmut_3: killed
    premerge.report.x_print_table__mutmut_4: killed
    premerge.report.x_print_table__mutmut_5: survived
    premerge.report.x_print_table__mutmut_6: killed
```

Figure 4.2 — Mutation testing results

Known gaps / future tests: the GitHub-Action comment poster is unit-tested with a mocked GitHub API but not against a live PR in CI; no performance benchmark on a 5k-commit repository; precision/recall validation across open-source repositories is planned; remaining survivors (mostly Rich colour/style strings and coupling cache internals) could be killed by snapshot and cache-corruption tests.

5. Appendix

a. Software Requirements Specification (SRS)

a.1 Introduction

Purpose. This SRS specifies PreMerge v0.1.0, a tool that predicts merge-conflict risk between git branches. **Intended audience:** developers, reviewers and course evaluators. **Scope:** CLI, GitHub Action and supporting configuration/caching; the web dashboard is out of scope for v1. **Definitions:** *hunk* — contiguous changed line range in a diff; *merge-base* — common ancestor of two branches; *coupling* — Jaccard similarity of two files' commit sets.

a.2 Overall description

Product perspective: stand-alone tool layered on the git CLI; optional GitHub Actions integration. **User classes:** developer, tech lead, DevOps engineer, researcher. **Operating environment:** Linux/macOS/Windows with Python 3.11+ and git, or Docker. **Constraints:** file/line-level analysis only; local branches by default; renames not tracked. **Assumptions:** full-history clone, append-only history, meaningful commits.

a.3 Specific requirements — functional

ID	Requirement	Priority
FR-1	Resolve two branches to their merge-base and diff each against it.	Must
FR-2	Report files modified by both branches (file-level overlap).	Must
FR-3	Report files whose changed line ranges intersect (line-level overlap); binary files are file-level only.	Must
FR-4	Compute pairwise historical coupling (Jaccard) from non-merge commit history.	Must
FR-5	Combine line overlap, file overlap and coupling into one weighted risk score in [0, 1].	Must
FR-6	Provide `premerge analyze` with modes: all pairs, one pair, one branch vs `--base`, all vs `--base`.	Must
FR-7	Rank pairs by descending risk and display a colour-coded table (red ≥ 0.5, yellow ≥ 0.2, green otherwise).	Must
FR-8	Generate a plain-English explanation per pair.	Should
FR-9	Emit JSON output with `--json`.	Should
FR-10	Cache coupling results in SQLite keyed by HEAD; `--no-cache` bypasses.	Should
FR-11	Load weights from `premerge.config.json` or `--config`; default 0.5/0.3/0.2.	Should
FR-12	Accept `--repo` to analyse a repository elsewhere on disk.	Should
FR-13	Run as a GitHub Action on `pull_request` events and post one report comment.	Must
FR-14	Update the existing PR comment in place (identified by marker) instead of posting duplicates.	Must
FR-15	Provide built-in `--help`.	Must

a.4 Non-functional requirements

ID	Category	Requirement
NFR-1	Performance	Cold analysis of a ~5k-commit repo < 10 s; cached re-run < 1 s; added CI time < 30 s per PR.
NFR-2	Usability	Zero-configuration default run; output names files, not only scores.
NFR-3	Portability	Pure Python + git; also available as Docker image.
NFR-4	Maintainability	Acyclic layered modules; core-logic test coverage > 80 % (achieved 96 %).
NFR-5	Reliability	Git failures raise `GitError` with the stderr text; cache can be bypassed.
NFR-6	Security / privacy	Read-only analysis; no network access except the Action's GitHub API call; token passed via environment.
NFR-7	Transparency	Published, tunable scoring formula; no black-box model.
NFR-8	Footprint	≤ 3 runtime dependencies (actual: 2).

a.5 External interfaces

User: CLI + Rich terminal table. **Software:** git CLI (subprocess); SQLite file `.premerge_cache.sqlite`; GitHub REST API (`/repos/{repo}/issues/{pr}/comments`). **Hardware:** none.

b. Data Flow Diagrams

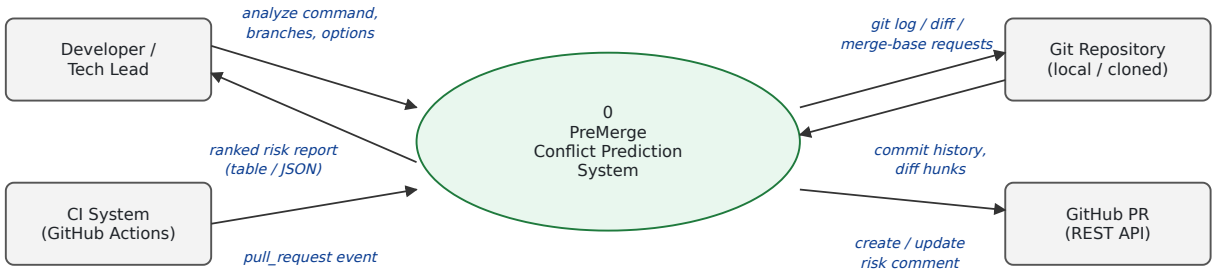


Figure A.1 — DFD Level 0 (context diagram)

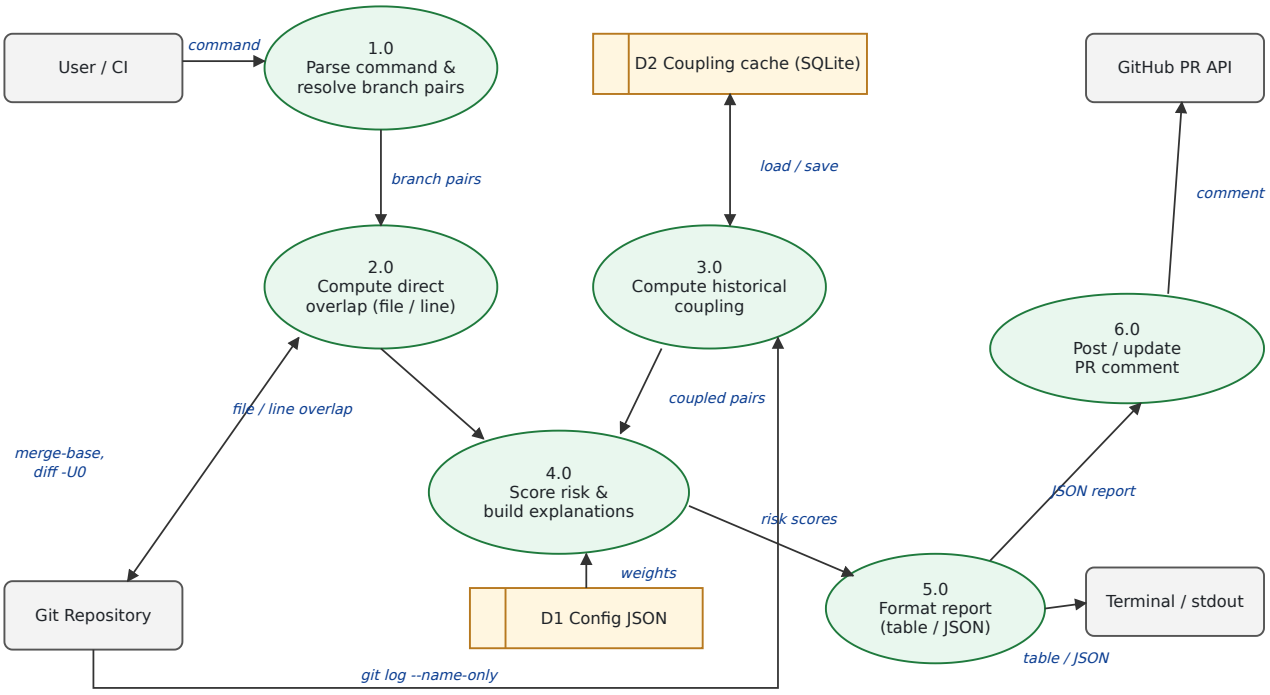


Figure A.2 — DFD Level 1

c. Entity-Relationship Diagram

PreMerge has no server database; the ER model describes the conceptual data it manipulates. Git entities (REPOSITORY, BRANCH, COMMIT, FILE_CHANGE) are read from git; COUPLING_CACHE is the only persisted table (SQLite `coupling_cache(head_hash, rev, file_counts, pair_counts, PRIMARY KEY(head_hash, rev))`); WEIGHTS_CONFIG is the JSON file; RISK_SCORE and PR_COMMENT are outputs.

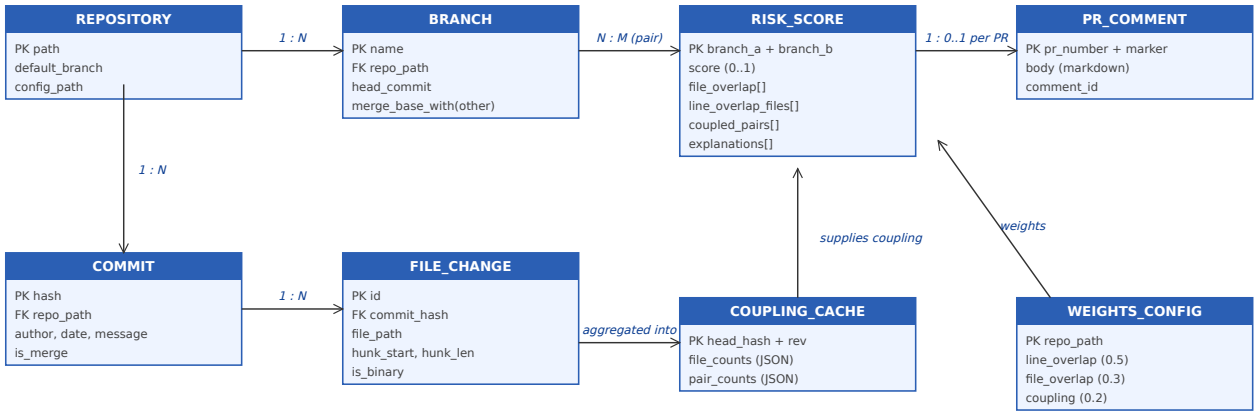


Figure A.3 — ER diagram

d. UML Diagrams

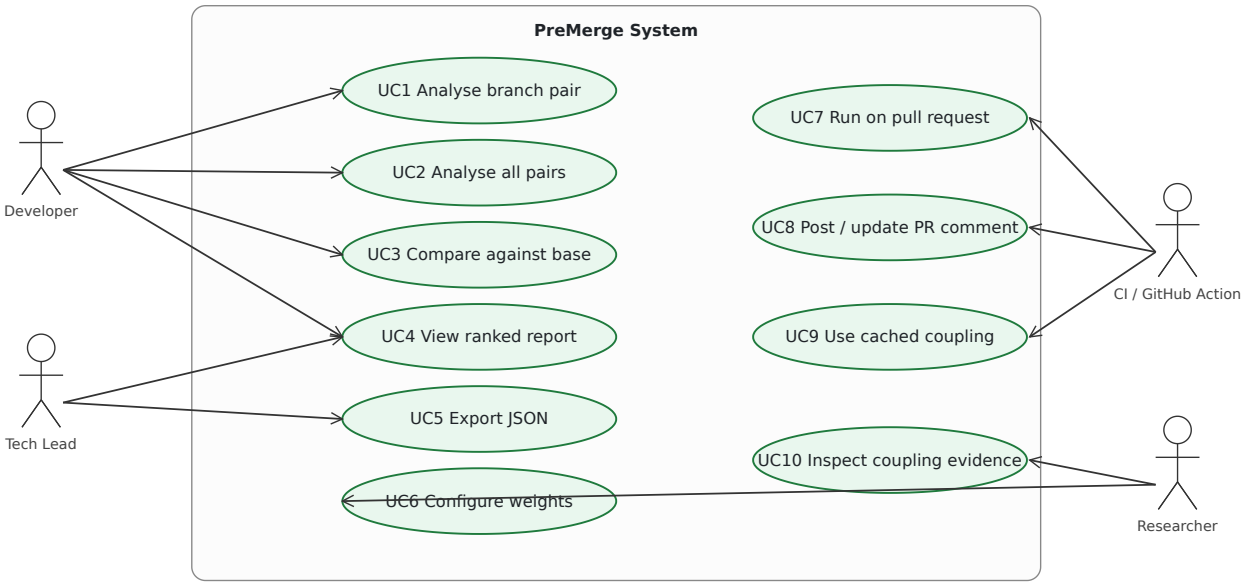


Figure A.4 — Use-case diagram

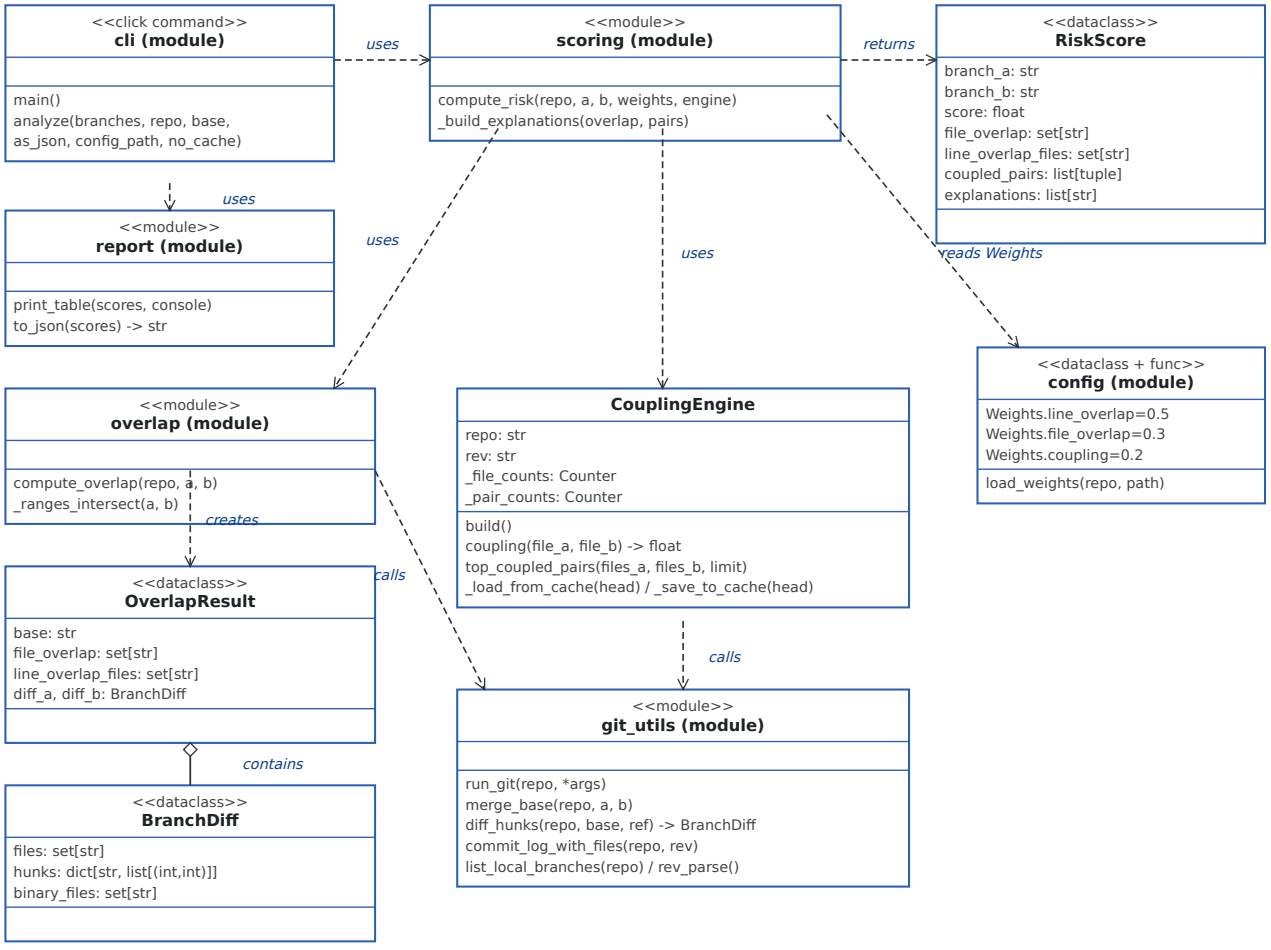


Figure A.5 — Class / module diagram

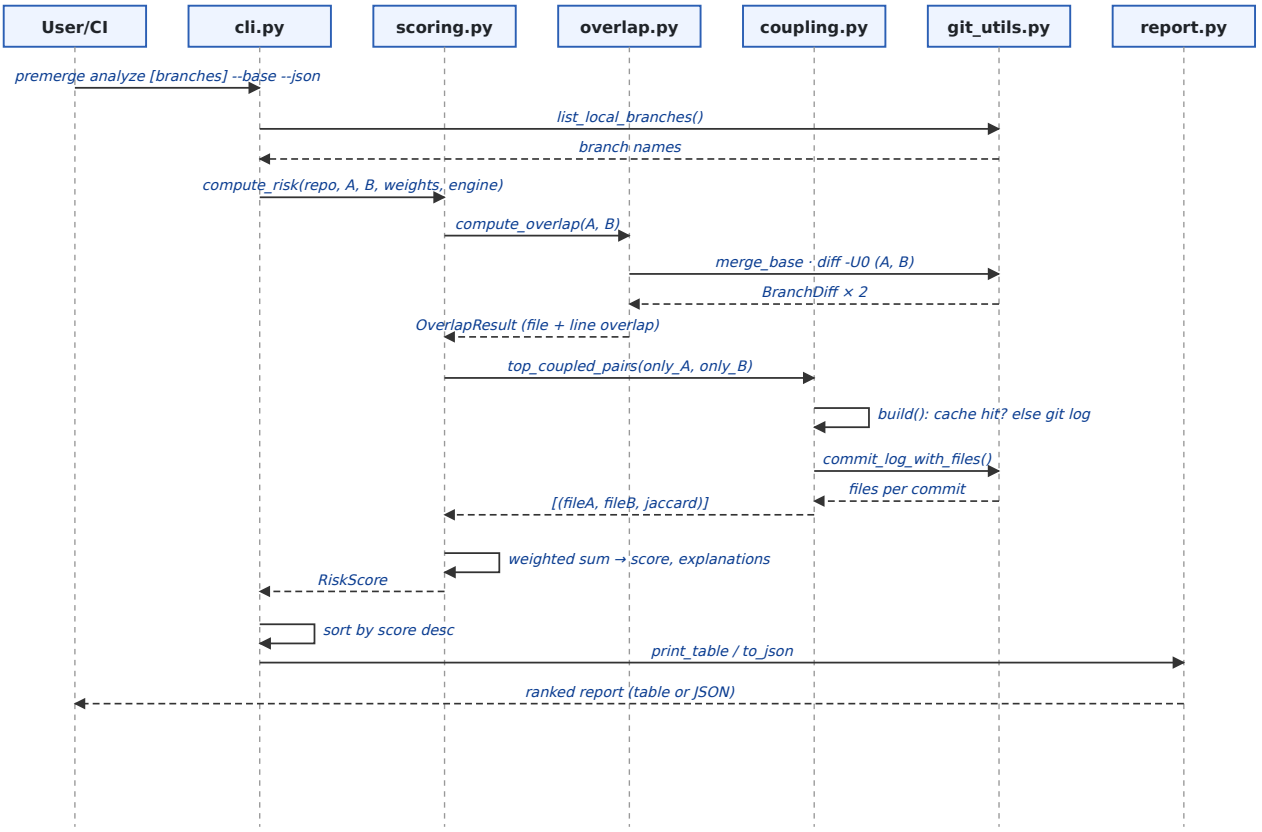


Figure A.6 — Sequence diagram (analyze run)

e. Code Listing / GitHub Link

GitHub repository: https://github.com/Waterbottles792/PreMerge
Contents: `premerge/` (core package), `github_action/`, `tests/`, `docs/`, `.github/workflows/`, `Dockerfile`, `pyproject.toml`. Core modules are listed in full below.

premerge/cli.py

```
"""premerge CLI entrypoint."""

from __future__ import annotations

import itertools

import click
from rich.console import Console

from . import git_utils
from .config import load_weights
from .coupling import CouplingEngine
from .report import print_table, to_json
from .scoring import compute_risk


@click.group()
def main() -> None:
    """PreMerge: predict merge conflicts before they happen."""


@main.command()
@click.argument("branches", nargs=-1)
@click.option("--repo", default=".", help="Path to the git repository.")
@click.option(
    "--base", default=None, help="Base branch to compare against (default: current branch)."
)
@click.option(
    "--json", "as_json", is_flag=True, help="Output machine-readable JSON instead of a table."
```

```
)
@click.option("--config", "config_path", default=None, help="Path to premerge.config.json.")
@click.option("--no-cache", is_flag=True, help="Skip the historical-coupling cache.")
def analyze(
    branches: tuple[str, ...],
    repo: str,
    base: str | None,
    as_json: bool,
    config_path: str | None,
    no_cache: bool,
) -> None:
    """Rank branch pairs by merge-conflict risk.

    With two BRANCHES given, analyzes just that pair. With one BRANCH given,
    analyzes it against --base. With none given and --base set, analyzes
    every other local branch against --base; with neither given, analyzes
    every pairwise combination of local branches (the useful default for
    "which of my branches are about to collide with each other").
    """
    console = Console()

    if len(branches) >= 2:
        pairs = [(branches[0], branches[1])]
    elif len(branches) == 1:
        base_branch = base or git_utils.current_branch(repo)
        pairs = [(base_branch, branches[0])]
    elif base is not None:
        other_branches = [b for b in git_utils.list_local_branches(repo) if b != base]
        pairs = [(base, b) for b in other_branches]
    else:
        pairs = list(itertools.combinations(git_utils.list_local_branches(repo), 2))

    if not pairs:
        console.print("[yellow]No other branches found to compare against.[/yellow]")
        return

    weights = load_weights(repo, config_path)
    coupling_engine = CouplingEngine(repo, rev="HEAD", use_cache=not no_cache)

    scores = [compute_risk(repo, a, b, weights, coupling_engine) for a, b in pairs]
    scores.sort(key=lambda s: s.score, reverse=True)

    if as_json:
        click.echo(to_json(scores))
    else:
        print_table(scores, console)


if __name__ == "__main__":
    main()
```

premerge/git_utils.py

```
"""Thin wrappers around the git CLI.

Everything PreMerge needs (log, diff, merge-base) is a subprocess call away,
so we shell out directly instead of taking on a GitPython dependency.
"""

from __future__ import annotations

import re
import subprocess
from dataclasses import dataclass, field


class GitError(RuntimeError):
    """Raised when a git subprocess invocation fails."""


def run_git(repo: str, *args: str) -> str:
    result = subprocess.run(
        ["git", "-C", repo, *args],
        capture_output=True,
        text=True,
    )
    if result.returncode != 0:
        raise GitError(f"git {' '.join(args)} failed: {result.stderr.strip()}")
    return result.stdout
```

```
def list_local_branches(repo: str) -> list[str]:
    out = run_git(repo, "for-each-ref", "--format=%(refname:short)", "refs/heads/")
    return [line.strip() for line in out.splitlines() if line.strip()]


def current_branch(repo: str) -> str:
    return run_git(repo, "rev-parse", "--abbrev-ref", "HEAD").strip()


def rev_parse(repo: str, ref: str) -> str:
    return run_git(repo, "rev-parse", ref).strip()


def merge_base(repo: str, ref_a: str, ref_b: str) -> str:
    return run_git(repo, "merge-base", ref_a, ref_b).strip()


def commit_log_with_files(repo: str, rev: str = "HEAD") -> list[list[str]]:
    """Return, per commit (newest first), the list of files it touched.

    Merge commits are skipped (--no-merges): their diff is usually the union
    of both parents' work and would artificially inflate coupling scores.
    """
    out = run_git(
        repo, "log", "--no-merges", "--name-only", "--pretty=format:COMMIT:%H", rev
    )
    commits: list[list[str]] = []
    current_files: list[str] | None = None
    for line in out.splitlines():
        if line.startswith("COMMIT:"):
            if current_files is not None:
                commits.append(current_files)
            current_files = []
        elif line.strip():
            assert current_files is not None
            current_files.append(line.strip())
    if current_files is not None:
        commits.append(current_files)
    return commits


_DIFF_GIT_RE = re.compile(r"^diff --git a/.+ b/(.+)$")
_HUNK_RE = re.compile(r"^@@ -(\d+)(?:,(\d+))? \+(\d+)(?:,(\d+))? @@")


@dataclass
class BranchDiff:
    """Changes on `ref` relative to `base` (typically their merge-base)."""

    files: set[str] = field(default_factory=set)
    # file -> list of (start_line, length) ranges touched, in the `ref` version
    hunks: dict[str, list[tuple[int, int]]] = field(default_factory=dict)
    binary_files: set[str] = field(default_factory=set)


def diff_name_only(repo: str, base: str, ref: str) -> list[str]:
    out = run_git(repo, "diff", "--name-only", base, ref)
    return [line.strip() for line in out.splitlines() if line.strip()]


def diff_hunks(repo: str, base: str, ref: str) -> BranchDiff:
    """Parse `git diff -U0 base ref` into per-file touched line ranges."""
    out = run_git(repo, "diff", "-U0", base, ref)
    result = BranchDiff()
    current_file: str | None = None
    for line in out.splitlines():
        match_file = _DIFF_GIT_RE.match(line)
        if match_file:
            current_file = match_file.group(1)
            result.files.add(current_file)
            result.hunks.setdefault(current_file, [])
        elif line.startswith("Binary files ") and current_file:
            result.binary_files.add(current_file)
        elif line.startswith("@@") and current_file:
            hunk_match = _HUNK_RE.match(line)
            if not hunk_match:
                continue
            start = int(hunk_match.group(3))
            # A "+c,0" hunk (pure deletion) has no "+" length; treat it as
            # touching a single line so it can still collide with an insertion
            # at that point in the other branch.
            length = int(hunk_match.group(4)) if hunk_match.group(4) is not None else 1
```

```
            result.hunks[current_file].append((start, max(length, 1)))
    return result
```

premerge/overlap.py

```
"""Direct-overlap detection between two branches relative to their merge-base."""

from __future__ import annotations

from dataclasses import dataclass, field

from . import git_utils
from .git_utils import BranchDiff


@dataclass
class OverlapResult:
    base: str
    file_overlap: set[str] = field(default_factory=set)
    line_overlap_files: set[str] = field(default_factory=set)  # subset of file_overlap
    diff_a: BranchDiff = field(default_factory=BranchDiff)
    diff_b: BranchDiff = field(default_factory=BranchDiff)


def _ranges_intersect(a: tuple[int, int], b: tuple[int, int]) -> bool:
    a_start, a_len = a
    b_start, b_len = b
    return a_start < b_start + b_len and b_start < a_start + a_len


def compute_overlap(repo: str, branch_a: str, branch_b: str) -> OverlapResult:
    """Diff both branches against their common ancestor and find overlap."""
    base = git_utils.merge_base(repo, branch_a, branch_b)
    diff_a = git_utils.diff_hunks(repo, base, branch_a)
    diff_b = git_utils.diff_hunks(repo, base, branch_b)

    file_overlap = diff_a.files & diff_b.files
    line_overlap_files: set[str] = set()
    for f in file_overlap:
        if f in diff_a.binary_files or f in diff_b.binary_files:
            # Can't compare line ranges in binaries; a shared touched binary
            # file is still a file-level risk, just not a line-level one.
            continue
        hunks_a = diff_a.hunks.get(f, [])
        hunks_b = diff_b.hunks.get(f, [])
        if any(_ranges_intersect(ha, hb) for ha in hunks_a for hb in hunks_b):
            line_overlap_files.add(f)

    return OverlapResult(
        base=base,
        file_overlap=file_overlap,
        line_overlap_files=line_overlap_files,
        diff_a=diff_a,
        diff_b=diff_b,
    )
```

premerge/coupling.py

```
"""Historical logical-coupling analysis: files that tend to change together.

coupling(A, B) = |commits touching both A and B| / |commits touching A or B|
(Jaccard similarity of the two files' change-sets across commit history.)
"""

from __future__ import annotations

import itertools
import json
import sqlite3
from collections import Counter
from pathlib import Path

from . import git_utils

CACHE_FILENAME = ".premerge_cache.sqlite"


class CouplingEngine:
    """Computes pairwise file coupling from commit history, with sqlite caching."""
```

```
    def __init__(self, repo: str, rev: str = "HEAD", use_cache: bool = True):
        self.repo = repo
        self.rev = rev
        self._use_cache = use_cache
        self._file_counts: Counter[str] | None = None
        self._pair_counts: Counter[tuple[str, str]] | None = None

    def _cache_path(self) -> Path:
        return Path(self.repo) / CACHE_FILENAME

    def _load_from_cache(self, head_hash: str) -> bool:
        if not self._use_cache or not self._cache_path().exists():
            return False
        conn = sqlite3.connect(self._cache_path())
        try:
            row = conn.execute(
                "SELECT file_counts, pair_counts FROM coupling_cache "
                "WHERE head_hash = ? AND rev = ?",
                (head_hash, self.rev),
            ).fetchone()
        finally:
            conn.close()
        if row is None:
            return False
        self._file_counts = Counter(json.loads(row[0]))
        self._pair_counts = Counter(
            {tuple(k.split("\x00")): v for k, v in json.loads(row[1]).items()}
        )
        return True

    def _save_to_cache(self, head_hash: str) -> None:
        if not self._use_cache:
            return
        assert self._file_counts is not None and self._pair_counts is not None
        conn = sqlite3.connect(self._cache_path())
        try:
            conn.execute(
                "CREATE TABLE IF NOT EXISTS coupling_cache ("
                "head_hash TEXT, rev TEXT, file_counts TEXT, pair_counts TEXT, "
                "PRIMARY KEY (head_hash, rev))"
            )
            pair_json = json.dumps(
                {f"{a}\x00{b}": v for (a, b), v in self._pair_counts.items()}
            )
            conn.execute(
                "INSERT OR REPLACE INTO coupling_cache VALUES (?, ?, ?, ?)",
                (head_hash, self.rev, json.dumps(self._file_counts), pair_json),
            )
            conn.commit()
        finally:
            conn.close()

    def build(self) -> None:
        """Build (or load from cache) the file co-occurrence counts.

        The cache key is just the current HEAD hash of `rev` — history is
        treated as append-only, so an unchanged HEAD means an unchanged cache.
        A rebase/force-push that rewrites history without moving HEAD would
        serve a stale entry; delete .premerge_cache.sqlite if that ever bites
        you.
        """
        head_hash = git_utils.rev_parse(self.repo, self.rev)
        if self._load_from_cache(head_hash):
            return

        file_counts: Counter[str] = Counter()
        pair_counts: Counter[tuple[str, str]] = Counter()
        for files in git_utils.commit_log_with_files(self.repo, self.rev):
            unique_files = sorted(set(files))
            for f in unique_files:
                file_counts[f] += 1
            for a, b in itertools.combinations(unique_files, 2):
                pair_counts[(a, b)] += 1

        self._file_counts = file_counts
        self._pair_counts = pair_counts
        self._save_to_cache(head_hash)

    def coupling(self, file_a: str, file_b: str) -> float:
        """Jaccard similarity of the two files' change-sets across history."""
        if self._file_counts is None:
```

```
            self.build()
        if file_a == file_b:
            return 1.0
        key = tuple(sorted((file_a, file_b)))
        together = self._pair_counts.get(key, 0)
        if together == 0:
            return 0.0
        either = self._file_counts[file_a] + self._file_counts[file_b] - together
        return together / either if either else 0.0

    def top_coupled_pairs(
        self, files_a: set[str], files_b: set[str], limit: int = 5
    ) -> list[tuple[str, str, float]]:
        """Coupling scores for cross pairs between two file sets, sorted desc."""
        if self._file_counts is None:
            self.build()
        scored = []
        for a in files_a:
            for b in files_b:
                if a == b:
                    continue
                score = self.coupling(a, b)
                if score > 0:
                    scored.append((a, b, score))
        scored.sort(key=lambda t: t[2], reverse=True)
        return scored[:limit]
```

premerge/scoring.py

```
"""Combine branch overlap + historical coupling into a single risk score.

risk(A, B) = w1 * line_overlap_score
           + w2 * file_overlap_score
           + w3 * avg_historical_coupling(files only one branch touched)
"""

from __future__ import annotations

from dataclasses import dataclass, field

from .config import Weights
from .coupling import CouplingEngine
from .overlap import OverlapResult, compute_overlap


@dataclass
class RiskScore:
    branch_a: str
    branch_b: str
    score: float
    file_overlap: set[str]
    line_overlap_files: set[str]
    coupled_pairs: list[tuple[str, str, float]]
    explanations: list[str] = field(default_factory=list)


def _build_explanations(
    overlap: OverlapResult, coupled_pairs: list[tuple[str, str, float]]
) -> list[str]:
    lines = []
    for f in sorted(overlap.line_overlap_files):
        lines.append(f"{f}: both branches edit overlapping lines")
    for f in sorted(overlap.file_overlap - overlap.line_overlap_files):
        lines.append(f"{f}: both branches touch this file (different lines)")
    for a, b, score in coupled_pairs:
        lines.append(
            f"{a} <-> {b}: historically change together (coupling={score:.2f}) "
            f"— one branch touched {a}, the other touched {b}"
        )
    return lines


def compute_risk(
    repo: str,
    branch_a: str,
    branch_b: str,
    weights: Weights,
    coupling_engine: CouplingEngine,
) -> RiskScore:
    overlap = compute_overlap(repo, branch_a, branch_b)
```

```
    touched_files = overlap.diff_a.files | overlap.diff_b.files
    file_overlap_score = len(overlap.file_overlap) / len(touched_files) if touched_files else 0.0
    line_overlap_score = (
        len(overlap.line_overlap_files) / len(overlap.file_overlap)
        if overlap.file_overlap
        else 0.0
    )

    # Hidden-coupling signal: files each branch touched that it did *not*
    # directly share with the other branch, checked for historical coupling.
    only_a = overlap.diff_a.files - overlap.file_overlap
    only_b = overlap.diff_b.files - overlap.file_overlap
    coupled_pairs = coupling_engine.top_coupled_pairs(only_a, only_b)
    coupling_score = (
        sum(score for _, _, score in coupled_pairs) / len(coupled_pairs)
        if coupled_pairs
        else 0.0
    )

    total = (
        weights.line_overlap * line_overlap_score
        + weights.file_overlap * file_overlap_score
        + weights.coupling * coupling_score
    )

    return RiskScore(
        branch_a=branch_a,
        branch_b=branch_b,
        score=round(total, 4),
        file_overlap=overlap.file_overlap,
        line_overlap_files=overlap.line_overlap_files,
        coupled_pairs=coupled_pairs,
        explanations=_build_explanations(overlap, coupled_pairs),
    )
```

premerge/config.py

```
"""Loads premerge.config.json, falling back to sensible defaults."""

from __future__ import annotations

import json
from dataclasses import dataclass
from pathlib import Path

CONFIG_FILENAME = "premerge.config.json"


@dataclass
class Weights:
    line_overlap: float = 0.5
    file_overlap: float = 0.3
    coupling: float = 0.2


def load_weights(repo: str, config_path: str | None = None) -> Weights:
    """Read `weights` from premerge.config.json if present, else use defaults."""
    path = Path(config_path) if config_path else Path(repo) / CONFIG_FILENAME
    if not path.exists():
        return Weights()
    data = json.loads(path.read_text())
    weights = data.get("weights", {})
    defaults = Weights()
    return Weights(
        line_overlap=weights.get("line_overlap", defaults.line_overlap),
        file_overlap=weights.get("file_overlap", defaults.file_overlap),
        coupling=weights.get("coupling", defaults.coupling),
    )
```

premerge/report.py

```
"""CLI output formatting: rich table for humans, JSON for machines."""

from __future__ import annotations

import json as json_module

from rich.console import Console
from rich.table import Table
```

```
from .scoring import RiskScore


def print_table(scores: list[RiskScore], console: Console | None = None) -> None:
    console = console or Console()
    if not scores:
        console.print("[yellow]No branch pairs to analyze.[/yellow]")
        return

    table = Table(title="PreMerge Conflict Risk Report")
    table.add_column("Branch A")
    table.add_column("Branch B")
    table.add_column("Risk", justify="right")
    table.add_column("Why")

    for s in scores:
        why = s.explanations[0] if s.explanations else "no overlap or coupling detected"
        if len(s.explanations) > 1:
            why += f" (+{len(s.explanations) - 1} more)"
        risk_style = "red" if s.score >= 0.5 else "yellow" if s.score >= 0.2 else "green"
        table.add_row(
            s.branch_a, s.branch_b, f"[{risk_style}]{s.score:.2f}[/{risk_style}]", why
        )

    console.print(table)

    for s in scores:
        if len(s.explanations) > 1:
            console.print(f"\n[bold]{s.branch_a} vs {s.branch_b}[/bold] (risk={s.score:.2f}):")
            for line in s.explanations:
                console.print(f"  - {line}")


def to_json(scores: list[RiskScore]) -> str:
    payload = [
        {
            "branch_a": s.branch_a,
            "branch_b": s.branch_b,
            "score": s.score,
            "file_overlap": sorted(s.file_overlap),
            "line_overlap_files": sorted(s.line_overlap_files),
            "coupled_pairs": [
                {"file_a": a, "file_b": b, "coupling": c} for a, b, c in s.coupled_pairs
            ],
            "explanations": s.explanations,
        }
        for s in scores
    ]
    return json_module.dumps(payload, indent=2)
```

github_action/post_comment.py

```
"""Posts (or updates) a PreMerge risk report as a PR comment.

Uses only the stdlib (urllib) against the GitHub REST API — no `requests`
dependency needed for a handful of HTTP calls.
"""

from __future__ import annotations

import json
import os
import urllib.error
import urllib.request

MARKER = "<!-- premerge-report -->"


def _api_request(url: str, token: str, method: str = "GET", data: dict | None = None):
    body = json.dumps(data).encode() if data is not None else None
    req = urllib.request.Request(url, data=body, method=method)
    req.add_header("Authorization", f"Bearer {token}")
    req.add_header("Accept", "application/vnd.github+json")
    req.add_header("Content-Type", "application/json")
    with urllib.request.urlopen(req) as resp:
        raw = resp.read()
        return json.loads(raw) if raw else None


def build_comment_body(report: list[dict]) -> str:
```

```
    if not report:
        return f"{MARKER}\n### PreMerge Conflict Risk Report\n\nNo other branches to compare — no risk
detected."

    lines = [
        MARKER,
        "### PreMerge Conflict Risk Report",
        "",
        "| Branch A | Branch B | Risk | Why |",
        "|---|---|---|---|",
    ]
    for entry in report:
        why = entry["explanations"][0] if entry["explanations"] else "no overlap or coupling detected"
        risk = entry["score"]
        emoji = "\U0001F534" if risk >= 0.5 else "\U0001F7E1" if risk >= 0.2 else "\U0001F7E2"
        lines.append(f"| {entry['branch_a']} | {entry['branch_b']} | {emoji} {risk:.2f} | {why} |")
    return "\n".join(lines)


def main() -> None:
    token = os.environ["GITHUB_TOKEN"]
    repo = os.environ["REPO"]
    pr_number = os.environ["PR_NUMBER"]
    report_path = os.environ.get("REPORT_PATH", "premerge_report.json")

    with open(report_path) as f:
        report = json.load(f)

    body = build_comment_body(report)
    comments_url = f"https://api.github.com/repos/{repo}/issues/{pr_number}/comments"

    try:
        existing = _api_request(comments_url, token) or []
    except urllib.error.HTTPError:
        existing = []
    bot_comment = next((c for c in existing if MARKER in c.get("body", "")), None)

    if bot_comment:
        update_url = f"https://api.github.com/repos/{repo}/issues/comments/{bot_comment['id']}"
        _api_request(update_url, token, method="PATCH", data={"body": body})
    else:
        _api_request(comments_url, token, method="POST", data={"body": body})


if __name__ == "__main__":
    main()
```

github_action/action.yml

```
name: "PreMerge Conflict Risk Check"
description: "Predicts merge-conflict risk between a PR's branch and its base by analyzing git history
coupling and live branch divergence, then comments on the PR."
inputs:
  github-token:
    description: "Token used to read/post the PR comment (usually secrets.GITHUB_TOKEN)."
    required: true
  base-ref:
    description: "Base branch to compare against."
    required: false
    default: ${{ github.event.pull_request.base.ref }}
  head-ref:
    description: "Head branch being merged."
    required: false
    default: ${{ github.event.pull_request.head.ref }}
runs:
  using: "composite"
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0
    - uses: actions/setup-python@v5
      with:
        python-version: "3.11"
    - name: Install PreMerge
      shell: bash
      run: pip install "premerge @ git+https://github.com/Waterbottles792/PreMerge.git"
    - name: Fetch base and head branches
      shell: bash
      run: git fetch origin "${{ inputs.base-ref }}" "${{ inputs.head-ref }}"
    - name: Run PreMerge analysis
      shell: bash
```

```
      run: >
        premerge analyze
        "origin/${{ inputs.base-ref }}"
        "origin/${{ inputs.head-ref }}"
        --json > premerge_report.json
    - name: Post PR comment
      shell: bash
      env:
        GITHUB_TOKEN: ${{ inputs.github-token }}
        REPORT_PATH: premerge_report.json
        REPO: ${{ github.repository }}
        PR_NUMBER: ${{ github.event.pull_request.number }}
      run: python "${{ github.action_path }}/post_comment.py"
```

tests/conftest.py

```
"""Synthetic git repo fixture, built via plain git CLI calls (no GitPython)."""

from __future__ import annotations

import subprocess
from pathlib import Path

import pytest


def run(repo: Path, *args: str) -> None:
    subprocess.run(["git", "-C", str(repo), *args], check=True, capture_output=True, text=True)


@pytest.fixture
def sample_repo(tmp_path: Path) -> Path:
    """A repo where auth.py and session.py always change together, and
    unrelated.py changes on its own — a clean fixture for coupling scores.
    """
    repo = tmp_path / "repo"
    repo.mkdir()
    run(repo, "init", "-q", "-b", "main")
    run(repo, "config", "user.email", "test@example.com")
    run(repo, "config", "user.name", "Test")

    for i in range(4):
        (repo / "auth.py").write_text(f"def login_v{i}(): pass\n")
        (repo / "session.py").write_text(f"def refresh_v{i}(): pass\n")
        run(repo, "add", "-A")
        run(repo, "commit", "-q", "-m", f"auth+session round {i}")

    (repo / "unrelated.py").write_text("x = 1\n")
    run(repo, "add", "-A")
    run(repo, "commit", "-q", "-m", "unrelated change")

    return repo
```

tests/test_cli_integration.py

```
"""End-to-end tests: drive the real `premerge analyze` CLI against synthetic repos."""

import json
import subprocess

from click.testing import CliRunner

from premerge.cli import main


def run(repo, *args: str) -> None:
    subprocess.run(["git", "-C", str(repo), *args], check=True, capture_output=True, text=True)


def _make_conflicting_branches(repo):
    run(repo, "checkout", "-q", "-b", "feature-a", "main")
    (repo / "auth.py").write_text("def login_v0(): pass\ndef new_a(): pass\n")
    run(repo, "commit", "-q", "-am", "a edits auth.py")
    run(repo, "checkout", "-q", "-b", "feature-b", "main")
    (repo / "auth.py").write_text("def login_v0(): pass\ndef new_b(): pass\n")
    run(repo, "commit", "-q", "-am", "b edits auth.py same spot")
    run(repo, "checkout", "-q", "main")
```

```
def _make_hidden_coupling_branches(repo):
    run(repo, "checkout", "-q", "-b", "touch-auth", "main")
    (repo / "auth.py").write_text("def login_v0(): pass\ndef extra(): pass\n")
    run(repo, "commit", "-q", "-am", "touch auth only")
    run(repo, "checkout", "-q", "-b", "touch-session", "main")
    (repo / "session.py").write_text("def refresh_v0(): pass\ndef extra(): pass\n")
    run(repo, "commit", "-q", "-am", "touch session only")
    run(repo, "checkout", "-q", "main")


def test_cli_pair_json_reports_line_overlap(sample_repo):
    _make_conflicting_branches(sample_repo)
    result = CliRunner().invoke(
        main, ["analyze", "feature-a", "feature-b", "--repo", str(sample_repo), "--json"]
    )
    assert result.exit_code == 0, result.output
    report = json.loads(result.output)
    assert len(report) == 1
    assert report[0]["line_overlap_files"] == ["auth.py"]
    assert report[0]["score"] >= 0.5


def test_cli_detects_hidden_coupling_without_file_overlap(sample_repo):
    _make_hidden_coupling_branches(sample_repo)
    result = CliRunner().invoke(
        main, ["analyze", "touch-auth", "touch-session", "--repo", str(sample_repo), "--json"]
    )
    assert result.exit_code == 0, result.output
    entry = json.loads(result.output)[0]
    assert entry["file_overlap"] == []
    assert entry["coupled_pairs"] and entry["coupled_pairs"][0]["coupling"] > 0
    assert entry["score"] > 0


def test_cli_all_pairs_ranked_highest_risk_first(sample_repo):
    _make_conflicting_branches(sample_repo)
    run(sample_repo, "checkout", "-q", "-b", "safe", "main")
    (sample_repo / "unrelated.py").write_text("x = 99\n")
    run(sample_repo, "commit", "-q", "-am", "safe change")
    result = CliRunner().invoke(main, ["analyze", "--repo", str(sample_repo), "--json"])
    assert result.exit_code == 0, result.output
    scores = [e["score"] for e in json.loads(result.output)]
    assert scores == sorted(scores, reverse=True)
    assert len(scores) == 6  # C(4,2) pairs over main, feature-a, feature-b, safe


def test_cli_base_mode_compares_every_branch_to_base(sample_repo):
    _make_conflicting_branches(sample_repo)
    result = CliRunner().invoke(
        main, ["analyze", "--base", "main", "--repo", str(sample_repo), "--json"]
    )
    assert result.exit_code == 0, result.output
    report = json.loads(result.output)
    assert {e["branch_b"] for e in report} == {"feature-a", "feature-b"}


def test_cli_table_output_renders_title_and_branches(sample_repo):
    _make_conflicting_branches(sample_repo)
    result = CliRunner().invoke(
        main, ["analyze", "feature-a", "feature-b", "--repo", str(sample_repo)]
    )
    assert result.exit_code == 0, result.output
    assert "PreMerge Conflict Risk Report" in result.output
    assert "feature-a" in result.output and "overlapping lines" in result.output


def test_cli_no_other_branches_message(sample_repo):
    result = CliRunner().invoke(
        main, ["analyze", "--base", "main", "--repo", str(sample_repo)]
    )
    assert result.exit_code == 0
    assert "No other branches" in result.output


def test_cli_custom_config_changes_score(sample_repo, tmp_path):
    _make_conflicting_branches(sample_repo)
    cfg = tmp_path / "cfg.json"
    cfg.write_text(json.dumps({"weights": {"line_overlap": 0.0, "file_overlap": 0.0, "coupling": 0.0}}))
    result = CliRunner().invoke(
        main,
        ["analyze", "feature-a", "feature-b", "--repo", str(sample_repo), "--json",
         "--config", str(cfg)],
```

```
    )
    assert json.loads(result.output)[0]["score"] == 0.0


def test_cli_warm_cache_gives_same_result_as_cold(sample_repo):
    _make_hidden_coupling_branches(sample_repo)
    args = ["analyze", "touch-auth", "touch-session", "--repo", str(sample_repo), "--json"]
    cold = CliRunner().invoke(main, args + ["--no-cache"]).output
    first = CliRunner().invoke(main, args).output   # writes cache
    warm = CliRunner().invoke(main, args).output    # reads cache
    assert cold == first == warm
```